Product Strategy and Pilot Plan

Developer-first distribution · Hybrid deployment · October 2026

> **Recommendation.** Build Eagle's Eye as a **developer-first Website Intelligence Engine**: a free self-hostable core plus a paid hosted service. Start with a pretrained open-weight model. Fine-tune a specialised one only once an evaluation dataset and paying users show it improves quality or cost.

1. The problem

Developers who build client sites keep rebuilding the same stack for an AI assistant: widget, crawler, index and retrieval, model and inference, conversation memory, business rules, configuration and monitoring. Hosted builders cut that work but add usage fees, vendor dependency and limits. Eagle's Eye sells **reduced engineering effort and operational complexity**. It does not sell “a chatbot”.

2. Positioning

Eagle's Eye lets developers turn any supported website into a configurable conversational assistant using a portable SDK, source-grounded knowledge, and a choice of hosted or self-hosted AI infrastructure.

Proposed differentiator	What it means
Portable runtime	Widget installable apart from the dashboard; documented APIs and adapters; supported self-hosting.
Website-to-assistant compilation	Versioned knowledge library, suggested intents, source-linked answers, testable config from minimal setup.
One config, two deployments	Same integration works hosted or self-hosted.
Verifiable answers	Citations, explicit uncertainty, deterministic rules, test suites and evaluation reports.

These are proposals, not claims, until built and validated. Lead with developer productivity and deployment flexibility. The proprietary model is a supporting advantage later.

3. The MVP

One customer journey

Install, enter a URL, crawl and index, test questions, get source-linked answers with sensible uncertainty, embed the widget on a separate site, run locally or on the hosted service.

Priority	Scope
P0 — launch	TypeScript SDK and light widget; secure project config; crawling, extraction, incremental indexing; BM25 retrieval with source metadata; model-provider abstraction and one working inference path; context and grounded answers; unknown-question fallback; hosted API and local dev deployment; docs and demo; security, rate limits, tests, usage metering.
P1 — after validation	Config dashboard; suggested intents and rules; knowledge versions and rollback; analytics and feedback; Docker Compose self-hosting; multiple projects; branding and domain limits; plan enforcement.
P2 — after evidence	Fine-tuned model; semantic retrieval and reranking; workflow builder; private enterprise deployment; more connectors; RBAC and audit; model routing.

Do not build first: a foundation model, a custom vector database, many integrations, complex billing analytics, a marketplace or a visual workflow builder. Use established components behind replaceable interfaces. Your durable value is the website compiler, conversation runtime, evaluation data, deployment experience and developer ecosystem.

4. Model strategy

Stage	Action
1. Launch	Serve a suitable open-weight model you control. Benchmark quality, latency, concurrency and cost before choosing hardware.
2. Evaluate	Build a domain dataset: questions, paraphrases, multi-turn chats, ambiguous and unanswerable cases, tool-use scenarios. Use reviewed synthetic data only.
3. Fine-tune	Parameter-efficient tuning for structured extraction, dialogue state, tool selection, concise grounded answers, abstention.
4. Optimise	Quantisation, batching, safe caching, small models for easy requests, larger models for hard ones when economics justify.

The model should specialise in operations the product repeats: interpreting website questions and follow-ups, extracting entities to validated schemas, selecting from supplied evidence, following configured policy, and recognising insufficient evidence. Website content is retrieved at run time, never memorised.

Hosted versus self-hosted

	Hosted	Self-hosted
Infrastructure	You manage	Customer manages
Model updates	Central	Customer chooses supported versions
Revenue	Subscription / usage	Licence, support, optional paid features
Data control	Depends on retention policy	Greater control
Support burden	Central operations	Documentation and deployment help

Track **cost per resolved conversation**: (inference + retrieval + infrastructure + support) divided by successfully resolved conversations. A cheap model that answers wrongly costs more through retries, escalation and churn. The first model milestone is competitive quality at sustainable cost, not parameter count.

5. Pricing

Open-core, with hypotheses to test. They are not confirmed willingness-to-pay.

Plan	Price	For
Community	$0	Experimentation: downloadable core, local dev, basic indexing.
Developer Pro	$19–$39/mo	Freelancers and agencies: multiple projects, hosted allowance, analytics.
Business Hosted	$49–$149/mo	Production sites: higher allowances, auto-reindexing, branding, monitoring, support.
Private / Enterprise	Custom	Private deployment, access controls, commitments, integrations.

Charge for what costs you money or gives recurring value: hosted inference, managed indexing, team management, evaluation and analytics, monitoring, support, private deployment. Do not charge a subscription just to download a library. Use predictable subscriptions with included allowances and clear overage, meter requests, tokens, indexed content, storage and crawls, and avoid unlimited hosted inference at launch.

Illustrative MRR (arithmetic, not forecast, before costs and churn): 50 × $29 = $1,450; 100 × $49 = $4,900; 250 × $79 = $19,750.

6. Go-to-market

- **GitHub and npm:** clean README, quick-start that works in minutes, live demo, typed SDK, example app, Docker instructions, architecture and security docs, public roadmap. Success is active projects, not stars.
- **Developer content:** tutorials tied to working repos, such as docs site to grounded assistant, React in one package, self-hosting, keeping answers in sync with site changes.
- **Agencies and freelancers:** multi-project management, reusable configs, client branding, a clear commercial licence. One agency on five sites beats five one-time installs.
- **Integrations on demand:** React and HTML first, then connectors that evaluating or paying users request.

7. Pilot plan: 90 days

Phase	Work	Success signal
Days 1–30 Validate	Interview 15–20 developers or agencies; fix MVP scope and licence; build crawler, index, retrieval and model adapter; test on real sites; simple demo.	Developers see it solve a real integration problem.
Days 31–60 Alpha	Publish SDK and widget; finish docs; hosted inference with limits; local path; recruit 5–10 design partners.	External developers integrate without your help.
Days 61–90 Monetise	Public beta; paid hosted plans; tutorials; better onboarding and diagnostics; track retention, cost, conversion; begin fine-tuning data.	Repeat deployments and first paying users.

These are targets, not delivery promises. Adjust to complexity and feedback.

Pricing validation interviews

Ask about current stack, integration and maintenance time, self-hosting need, present spend, switching triggers, which features justify payment, and licence versus subscription versus usage preference. Then offer real pilot plans at different prices. Paid pilots and continued use outweigh survey enthusiasm.

8. Metrics and targets

Metric	Why it matters
Time to first answer / first integration	Onboarding and developer experience
Install-to-activation rate	Whether downloads become projects
Weekly active projects; 30-day retention	Sustained value
Answers with supported evidence; correctness	Core value and trust
Unknown and escalation rates	Knowledge and capability gaps
Free-to-paid conversion; MRR	Monetisation and traction
Gross margin per hosted project	Viability of model serving
Support time per customer	Whether it is truly easy to use

Beta targets: 5–10 active design partners; at least five independent integrations; measured time-to-first-answer and time-to-production; a supported and unsupported question test set; first paid pilots before major inference spend; documented reasons for activation failure.

9. Risks and mitigations

Risk	Mitigation
Looks like another chatbot builder	Lead with portable SDK, website compilation, transparent retrieval, deployment flexibility, measurable accuracy; validate with developers.
Model hosting too costly	Benchmark first; usage limits; batching; add hardware only when paid demand supports it.
Fluent but wrong answers	Authoritative sources, confidence thresholds, validation, explicit abstention, repeatable evaluations.
Self-hosting support load	Few tested profiles, health diagnostics, published hardware needs, easy hosted option.
Downloads without payment	Keep the core useful; charge for hosted value, convenience, premium features, support; test payment early.

10. Build order and success test

1. Core engine: ingestion, retrieval, reliable conversation.
2. Developer product: SDK, widget, docs, examples.
3. Hosted service: your model, managed indexing, usage controls, subscriptions.
4. Specialised model, only after an evaluation set shows it helps.
5. Enterprise offering once customers ask.

> **The test:** can an independent developer integrate Eagle's Eye into a real website, get trustworthy answers from its content, and maintain the assistant with far less effort than building it themselves? If yes, and some will pay for hosted convenience, you have a credible foundation.

Prepared 11 October 2026. Prices and targets are hypotheses to test. Competitor references are from supplied material and unverified.